

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
З ЛАБОРАТОРНОЇ РОБОТИ № 1
з дисципліни «Професійний Python»
на тему: «Організація професійного Python-проєкту та Git»

Виконав:
студент 3 курсу
групи ФЕП-33
Франков Дмитро
Олександрович

Перевір:
професор Свелеба Сергій
Андрійович

Львів-2026

Мета роботи

Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл `pyproject.toml`, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

Номер індивідуального варіанта

Варіант 15.

Постановка задачі

Створити консольний застосунок для обліку результатів наукових експериментів.

Для сутності `Experiment` зберігати:

- назва;
- `researcher` (дослідник);
- параметр;
- результат;
- дата.

Реалізувати функції:

- додавання `Experiment`;
- пошук за `researcher`;
- середнє значення результатів;
- знаходження `minimum`/`maximum` результатів;
- фільтрацію результатів за заданим числовим діапазоном (із сортуванням).

Проєкт має використовувати `src-layout`, типізацію (`type hints`), щонайменше три модулі, точку входу з конструкцією `if __name__ == "__main__":`, містити конфігурацію `pyproject.toml`, `.gitignore`, `README.md` та історію комітів у Git.

Структура проєкту

None

```
PPython/  
├── .gitignore  
├── pyproject.toml  
├── README.md  
└── src/  
    ├── lab_logging/  
    │   ├── __init__.py  
    │   ├── main.py  
    │   ├── models.py  
    │   └── services.py
```

Опис створених модулів

- **src/lab_logging/__init__.py:** позначає каталог як Python-пакет і фіксує версію модуля (`__version__ = "0.1.0"`).
- **src/lab_logging/models.py:** описує модель даних Experiment за допомогою `@dataclass` із типізованими полями `name: str`, `researcher: str`, `parameter: str`, `result: float`, `date: str`.
- **src/lab_logging/services.py:** модуль бізнес-логіки з чистими функціями обробки списків експериментів (`add_experiment`, `find_by_research`, `average_result`, `min_max_result`, `filter_by_range`).
- **src/lab_logging/main.py:** консольна точка входу застосунку. Створює початкові тестові дані, викликає методи сервісного шару та виводить результат у відформатованому табличному вигляді. Запуск контролюється перевіркою `if __name__ == "__main__":`.

Текст основних фрагментів коду
src/lab_logging/models.py

```
Python
from dataclasses import dataclass
from datetime import date

@dataclass
class Experiment:
    name      : str
    researcher : str
    parameter  : str
    result     : float
    date       : date
```

src/lab_logging/services.py

```
Python
from .models import Experiment

def add_experiment(
    experiments: list[Experiment],
    experiment: Experiment,
) -> None:
    """Add an experiment to the experiments list"""
    experiments.append(experiment)
```

```

def find_by_research(
    experiments: list[Experiment],
    researcher: str,
) -> list[Experiment]:
    """Find an experiment in the experiments list"""
    query = researcher.lower().strip()
    return [
        exp
        for exp in experiments
        if query in exp.researcher.lower()
    ]

def average_result(
    experiments: list[Experiment],
) -> float:
    """Calculate the average result of the experiments"""
    if not experiments:
        return 0.0

    total = 0
    for exp in experiments:
        total += exp.result

    total = total / len(experiments)

    return total

def min_max_result(
    experiments: list[Experiment],
) -> tuple[Experiment, Experiment]:
    """Calculate the minimum and maximum result of the experiments"""
    if not experiments:
        return None

    min_exp = min(experiments, key=lambda x: x.result)
    max_exp = max(experiments, key=lambda x: x.result)

    return min_exp, max_exp

```

```
def filter_by_range(
    experiments: list[Experiment],
    start: float,
    end: float,
) -> list[Experiment]:
    """Filter experiments by the range sorted by result"""
    return sorted(
        [exp
         for exp in experiments
         if start <= exp.result <= end],
        key=lambda exp: exp.result,
    )
```

Пояснення pyproject.toml

Файл стандарту PEP 518/621, що описує конфігурацію складання проєкту:

- **[build-system]:** використовує `setuptools` як рушій збірки (build-backend).
- **[project]:** задає метадані пакету — ім'я `lab-logging`, початкову версію `0.1.0` та вимогу до інтерпретатора `requires-python = ">=3.10"`.
- **[project.scripts]:** реєструє точку виклику з консолі `lab-logging = "lab_logging.main:main"`.
- **[tool.setuptools.packages.find]:** налаштовує сканування теки `src/` для автоматичного знаходження пакетів під час `editable installation` (`pip install -e .`).

Вміст .gitignore

Файл виключає з індексу Git системне сміття, файли середовища та білди:

```
None
.venv/
__pycache__/
*.pyc
.pytest_cache/
.idea/
.vscode/
dist/
build/
*.egg-info/
.env
```

9. Фрагмент [README.md](#)

Облік наукових експериментів

Опис

Консольний проєкт на Python 3 для обліку наукових експериментів.

Кожен експеримент містить такі поля:

- **Назва** (`name``)
- **Дослідник** (`researcher``)
- **Параметр** (`parameter``)
- **Результат** (`result``)
- **Дата** (`date``)

Проект реалізує такі операції:

- додавання експерименту;
- пошук експериментів за дослідником;
- обчислення середнього результату;
- пошук мінімального та максимального результатів;
- фільтрація експериментів за діапазоном результатів.

Screenshots запуску application

```
(.venv) PS C:\Users\frankov\Documents\projects\labs\PPython\Lab_01> lab-logging
```

```
=== Початковий список експериментів ===
```

Назва	Дослідник	Параметр	Результат	Дата
Thermal Expansion	Oleh Koval	Delta L (mm)	2.45	2026-03-10
Tensile Strength	Iryna Melnyk	Stress (MPa)	450.00	2026-03-12
Thermal Conductivity	Oleh Koval	Watts/(m*K)	15.80	2026-03-15

1. Додавання нового експерименту:

Додано: Electrical Resistance

2. Пошук експериментів дослідника 'Koval':

Назва	Дослідник	Параметр	Результат	Дата
Thermal Expansion	Oleh Koval	Delta L (mm)	2.45	2026-03-10
Thermal Conductivity	Oleh Koval	Watts/(m*K)	15.80	2026-03-15

3. Середнє значення результатів: 147.19

4. Мінімум: Thermal Expansion (2.45) | Дослідник: Oleh Koval

Максимум: Tensile Strength (450.0) | Дослідник: Iryna Melnyk

5. Фільтрація результатів у діапазоні [10.0; 200.0] (відсортовано):

Назва	Дослідник	Параметр	Результат	Дата
Thermal Conductivity	Oleh Koval	Watts/(m*K)	15.80	2026-03-15
Electrical Resistance	Anna Shevchenko	Resistance (Ohm)	120.50	2026-03-18

Результат git status

```
(.venv) PS C:\Users\frankov\Documents\projects\labs\PPython\Lab_01> git status
```

```
On branch main
```

```
Your branch is up to date with 'origin/main'.
```

```
nothing to commit, working tree clean
```

Результат git log --oneline

```
(.venv) PS C:\Users\frankov\Documents\projects\labs\PPython\Lab_01> git log --oneline
```

```
1d7a484 (HEAD -> main, origin/main) Initialize lab_logging package with versioning
```

```
6c95137 Remove old project files and add new structure for experiment tracking application
```

```
59c8de4 Merge branch 'main' of https://github.com/FrankovDmutro/PPython
```

```
088b38d Add README.md with project description, requirements, installation instructions, and project structure
```

```
0cec5ba Add README.md with project description, requirements, installation instructions, and project structure
```

```
3ec4b1f Refactor main.py to implement demo experiments and enhance output formatting; update .gitignore and add pyproject.toml for project configuration
```

```
3c18b59 Add checks for empty experiments list in average_result and min_max_result functions
```

```
5056d15 Implement Experiment model and add services for managing experiments
```

```
324fef9 Add initial project structure with .gitignore, README, pyproject.toml, and source files
```

```
e425f85 Add initial project structure with .gitignore, README, pyproject.toml, and source files
```

Відповіді на контрольні запитання

1. Що таке Python-проект?

Структурована система з вихідного коду, модулів, залежностей та конфігураційних файлів, підготовлена для розробки, запуску і дистрибуції.

2. Чим проєкт відрізняється від одного Python script?

Скрипт виконує лінійну задачу в одному файлі; проєкт має модульну архітектуру, ізоляцію залежностей, документацію та конфігурацію складання.

3. Що таке Python module?

Файл із розширенням .py, який містить визначення функцій, класів чи змінних для подальшого імпорту.

4. Що таке Python package?

Папка, яка містить файл `__init__.py` та модулі, що дозволяє організувати простір імен за ієрархією.

5. Для чого потрібний `__init__.py`?

Ініціалізує пакет, робить каталог видимим для модуля імпорту та контролює експорт інтерфейсів пакета.

6. Що таке `src-layout`?

Структура розташування коду, де вихідні пакети винесені в окрему підпапку `src/`, а не в корінь репозиторію.

7. Які переваги має `src-layout`?

Запобігає імпорту локальних файлів замість встановленого пакета та гарантує коректне тестування встановленої збірки.

8. Для чого використовується `virtual environment`?

Для повної ізоляції бібліотек та версій пакетів одного проєкту від інших та від глобальної системи.

9. Як створити `.venv`?

Командою `python -m venv .venv`.

10. Як активувати `virtual environment`?

Windows: `.venv\Scripts\activate`; Linux/macOS: `source .venv/bin/activate`.

11. Для чого використовується `pip`?

Для завантаження, оновлення та видалення сторонніх бібліотек з репозиторію PyPI.

12. Чому рекомендується `python -m pip`?

Забезпечує виклик `pip`, прив'язаного до конкретного активного інтерпретатора Python.

13. Для чого потрібний `pyproject.toml`?

Єдиний декларативний конфігураційний файл складання, залежностей та налаштувань інструментів Python-проєкту.

14. Що містить секція `[project]`?

Основні метадані: назву, версію, залежності, опис та мінімальну сумісну версію Python.

15. Що таке `editable installation`?

Режим інсталяції `pip install -e .`, що додає посилання на поточний каталог розробки без фізичного копіювання файлів у `site-packages`.

16. Для чого використовується `if __name__ == "__main__":`?

Запобігає автоматичному запуску коду модулю під час його імпортування в інші файли.

17. Що таке Git?

Децентралізована система контролю версій для відстеження змін у файлах та спільної розробки.

18. Що таке repository?

Сховище проєкту під керуванням Git разом з усією історією комітів, тегів і гілок у папці `.git`.

19. Що таке commit?

Знімок стану заіндексованих файлів репозиторію в конкретний момент часу з описом змін.

20. Для чого використовується git status?

Показує стан файлів: чи є невідстежувані, змінені чи підготовлені до коміту елементи.

21. Для чого git add?

Додає обрані файли або директорії до зони підготовки (staging area) перед комітом.

22. Для чого git commit?

Фіксує підготовлені зміни зі staging area в постійну історію репозиторію.

23. Що показує git log?

Повний список створених комітів у хронологічному порядку із зазначенням автора, дати й хешу.

24. Для чого потрібен .gitignore?

Задає список масок файлів і папок, які Git має повністю ігнорувати і не пропонувати до коміту.

25. Чому .venv не потрібно комітити?

Каталог містить платформозалежні бінарники великого розміру; оточення завжди генерується наново на цільовій машині.

26. Для чого потрібний README?

Первинна документація, яка пояснює призначення коду, способи його інсталяції та запуску.

27. Що таке type hints?

Явні позначення типів параметрів функцій і змінних для підвищення читабельності коду та статичного аналізу.

28. Для чого програму розділяють на modules?

Для структуризації архітектури, ізоляції зон відповідальності та спрощення тестування.

29. Що таке business logic?

Набір основних правил, формул і функцій обробки даних предметної області проєкту.

30. Чому потрібно відокремлювати application entry point від business logic?

Для збереження можливості повторного використання логіки (наприклад, у тестах або веб-сервісах) незалежно від інтерфейсу вводу/виводу.

Висновок

У ході лабораторної роботи було створено повноцінний структурований Python-проект за принципом src-layout. Налаштовано ізольоване віртуальне середовище, розроблено модель Experiment на базі @dataclass та реалізовано незалежний модуль бізнес-логіки з анотаціями типів. Проект сконфігуровано через ruroject.toml з підтримкою CLI-запуску, задокументовано у README.md, захищено від зайвих артефактів через .gitignore та збережено за допомогою логічних етапів у системі контролю версій Git.